Web Security Landscape

Kameswari Chebrolu

Department of CSE,
IIT Bombay

https://simonsayszero.files.wordpress.com/2018/07/a7-1.png?w=640

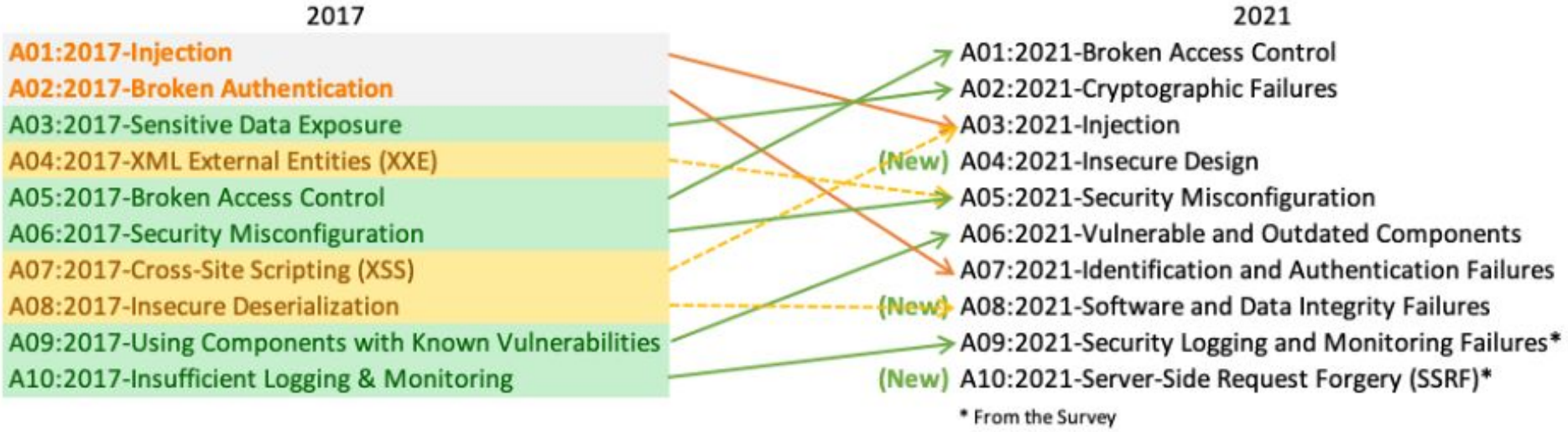
Background

- Web development a very popular profession!
- Involves creating and maintaining websites or web applications
  - Lots goes behind the scenes, as we saw in background!
- Web development can introduce various attack vectors that can be exploited by malicious players
  - Lots goes here as well, as we saw in server/client side attacks

OWASP Top 10

- Increasing threats and vulnerabilities in web applications → Open Web Application Security Project (OWASP)
  - A nonprofit organization focused on improving software security
  - Aim: Provide valuable resources, tools, and best practices to help organizations enhance security

- OWASP Top 10: document that outlines ten most critical web application risks
  - Regularly updated every few years
  - Based on analysis of vulnerability data spanning hundreds of organisations and 100,000+ real world applications
  - Vulnerabilities assessed using several factors: Detectability, exploitability, and potential impact
  - Helps organizations to prioritize and address common security issues!



https://owasp.org/Top10/

Comparison Between 2021-2025

OWASP-2021	OWASP-2025
A01:2021 - Broken Access Control	A01:2025 - Broken Access Control
A02:2021 - Cryptographic Failures	A02:2025 - Security Misconfiguration (NEW)
A03:2021 - Injection	A03:2025 - Software Supply Chain Failures
A04:2021 - Insecure Design	A04:2025 - Cryptographic Failures
A05:2021 - Security Misconfiguration	A05:2025 - Injection
A06:2021 - Vulnerable and Outdated Components	A06:2025 - Insecure Design
A07:2021 - Identification and Authentication Failures	A07:2025 - Authentication Failures
A08:2021 - Software and Data Integrity Failures	A08:2025 - Software or Data Integrity Failures
A09:2021 - Security Logging and Monitoring Failures	A09:2025 - Logging & Alerting Failures
A10:2021 - Server-Side Request Forgery (SSRF)	(NEW) A10:2025 - Mishandling of Exceptional Conditions

Source : OWASP Top 10:2025 RC1

A01:2025 – “Broken Access Control”

- Resources not properly protected
  - Information is inadvertently exposed to unauthorized parties
- SSRF (“Server-Side Request Forgery”) now folded into this category
- In the dataset ~3.73% of tested applications had at least one mapped weakness
- E.g. IDORs, Path Traversal, simple mis-applied permissions on data/services, SSRF via internal services, missing tamper-proofing of role checks
- Checklist:
  - Ensure role/permission matrices are enforced server-side (not only client-side).
  - Verify internal APIs/services cannot be invoked inadvertently by external users.
  - Review SSRF-capable endpoints (e.g., proxy, fetcher, URL parameter) and ensure they validate white listed hosts
  - Monitor for “excess privileges” and review for orphaned access rights

- A02:2025 – “Security Misconfiguration”: misconfiguration ranging from OS/Cloud-platform to applications
  - 3.00% of applications had one or more of these vulnerabilities
  - Modern applications increasingly rely on configuration
    - Deployment, cloud, containers, feature toggles, infrastructure as code etc
    - A minor mis-setting (e.g., public bucket, default credential) can result in massive exposure
  - E.g. Default accounts/credentials, unnecessary enabled features (e.g., debug endpoints in production), open cloud-storage buckets, insecure HTTP headers, unpatched/unhardened chassis, inadvertently exposed services or ports, verbose error display etc
  - Checklist:
    - Harden deployment pipelines: remove default credentials, disable unused endpoints
    - Review IaC (Infrastructure as Code) templates & cloud-permissions for unnecessary public exposure.
    - Ensure secure headers (CSP, HSTS, etc) are used and configuration is validated post-deployment
    - Implement configuration scanning and compliance checks as part of CI/CD.

HTTP Strict Transport Security (HSTS)

- A browser security policy that forces HTTPS only
  - Prevents use of insecure HTTP connections
- Strict-Transport-Security: max-age=31536000; includeSubDomains
  - max-age → duration browser remembers (in seconds)
  - includeSubDomains → applies to all subdomains

Step 1: User visits HTTP
http://example.com

Step 2: Server responds with redirect
Server sends:
HTTP/1.1 301 Moved Permanently
Location: https://example.com
HSTS is NOT applied yet, because the connection is still HTTP (untrusted).

Step 3: Browser follows redirect → HTTPS
https://example.com

Step 4: Server sends HSTS header (over HTTPS)
Strict-Transport-Security: max-age=31536000; includeSubDomains

Why HSTS is needed?

- Prevents SSL Stripping attacks
- Protects against Man-in-the-Middle (MITM)
- Ensures secure communication always

Key Limitation: First visit still vulnerable (unless preloaded)

- What is SSL Stripping?
  - A Man-in-the-Middle (MITM) attack
  - Downgrades HTTPS → HTTP
- Attack Flow (without HSTS)
  - User enters: http://example.com
  - Server sends redirect: 301 → https://example.com
  - Attacker intercepts and removes redirect → User continues over HTTP (insecure)
  - Attacker reads/modifies data (passwords, cookies)
- With HSTS Enabled
  - Browser already knows: HTTPS only
  - Request becomes https://example.com
  - NoteL First visit still vulnerable
    - But solved by HSTS Preload (A built-in list of websites inside browsers)

A03:2025 – Software Supply Chain Failures

- A new category expanded from “Vulnerable and Outdated Components”
- Covers failures in the software ecosystem: dependencies, build systems, distribution infrastructure, package management, and trust in those components
- Testing data is still limited (few apps flagged)
  - Exploit impact and community concern are high
- E.g. Use of a compromised library, malicious update injected into build pipeline, dependency tree with unchecked transitive risks, compromised distribution channel
- Checklist:
  - Maintain a software bill of materials (SBOM) and map all dependencies (direct & transitive).
  - Validate integrity of build pipelines (CI/CD) and ensure updates are signed/verified
  - Monitor third-party component alerts and have patch/replace process for high-risk dependencies

Comparison Between 2021-2025

OWASP-2021	OWASP-2025
A01:2021 - Broken Access Control	A01:2025 - Broken Access Control
A02:2021 - Cryptographic Failures	A02:2025 - Security Misconfiguration (NEW)
A03:2021 - Injection	A03:2025 - Software Supply Chain Failures
A04:2021 - Insecure Design	A04:2025 - Cryptographic Failures
A05:2021 - Security Misconfiguration	A05:2025 - Injection
A06:2021 - Vulnerable and Outdated Components	A06:2025 - Insecure Design
A07:2021 - Identification and Authentication Failures	A07:2025 - Authentication Failures
A08:2021 - Software and Data Integrity Failures	A08:2025 - Software or Data Integrity Failures
A09:2021 - Security Logging and Monitoring Failures	A09:2025 - Logging & Alerting Failures
A10:2021 - Server-Side Request Forgery (SSRF)	(NEW) A10:2025 - Mishandling of Exceptional Conditions

Source : OWASP Top 10:2025 RC1

A04:2025 – “Cryptographic Failures”: Failures related to cryptography

- Failures around encryption, key management, secure channel usage, or cryptographic algorithms
  - Can lead to sensitive data exposure or system compromise
- ~3.80% of tested applications had one or more of these vulnerabilities
- E.g. Weak encryption algorithms, poor key generation, not verifying certificates, simple lack of transport layer protection (e.g., HTTP vs HTTPS), crypto keys in source code repositories, insecure random number generation, storing sensitive data unencrypted
- Checklist:
  - Ensure sensitive data is encrypted at rest and in transit; review algorithm and key-length appropriateness
  - Validate key lifecycle: creation, rotation, retirement, access control
  - Check for usage of obsolete algorithms or libraries flagged by modern cryptographic standards
  - Verify via code reviews that crypto primitives are not being mis-used (e.g., home-grown crypto)

A05:2025 – “Injection”: inject malicious code into data input fields

- Though ranked #5 in 2025 (down from earlier positions), injection is still highly exploited
- Leads to unauthorized access or manipulation of data.
- Covers SQl injection, XSS, command injection, code injection
- Checklist:
  - Validate all user input; use parameterized queries or safe APIs rather than string-concatenation
  - Escaping/encoding output based on context (HTML, JavaScript, URL, SQL)
  - Have active dynamic scanning (DAST) + static analysis with injection detection rules

A06:2026 – “Insecure Design”: risks related to architecture and design flaws

- a very broad category
- E.g. Lack of threat modelling, insecure defaults by design, business-logic vulnerabilities (e.g., price manipulation, discount escalation)
- Checklist:
  - Incorporate threat modelling and security architecture review in project planning
  - Define and document trust boundaries, data flow diagrams, misuse cases.
  - Review business logic for abnormal flows (e.g., discount stacking, role escalation)
  - Ensure security requirements (not just functional) are part of UI/UX and backend design

A07:2025 – Authentication Failures

- Can lead to unauthorized access to accounts and sensitive data
- Relates to vulnerabilities in authentication mechanisms, session management, identity life-cycle, and improper credential handling
- E.g. Weak password policy, missing MFA for sensitive actions, page accessible without login, insecure credential storage
- Checklist:
  - Use robust authentication libraries; avoid custom home-built auth implementations
  - Enforce MFA for privileged or sensitive operations
  - Review session-management (timeout, invalidation after logout, secure cookies)
  - Ensure credential storage uses best practices (salted hashes, no plaintext).

Comparison Between 2021-2025

OWASP-2021	OWASP-2025
A01:2021 - Broken Access Control	A01:2025 - Broken Access Control
A02:2021 - Cryptographic Failures	A02:2025 - Security Misconfiguration (NEW)
A03:2021 - Injection	A03:2025 - Software Supply Chain Failures
A04:2021 - Insecure Design	A04:2025 - Cryptographic Failures
A05:2021 - Security Misconfiguration	A05:2025 - Injection
A06:2021 - Vulnerable and Outdated Components	A06:2025 - Insecure Design
A07:2021 - Identification and Authentication Failures	A07:2025 - Authentication Failures
A08:2021 - Software and Data Integrity Failures	A08:2025 - Software or Data Integrity Failures
A09:2021 - Security Logging and Monitoring Failures	A09:2025 - Logging & Alerting Failures
A10:2021 - Server-Side Request Forgery (SSRF)	(NEW) A10:2025 - Mishandling of Exceptional Conditions

Source : OWASP Top 10:2025 RC1

A08:2021 – “Software and Data Integrity Failures”

- Focuses on failures to verify integrity of code, libraries, data, or configurations within the system
- E.g. Modern CI/CD pipelines include auto-update functionality → corrupted payloads can be deployed and executed
- E.g. Unsigned software updates, deserialization of untrusted data, manipulation of configuration files, loading of unverified plugins
- Checklist:
  - Verify digital signatures and checksums in update mechanisms
  - Restrict deserialization of untrusted data or use safe serializers
  - Audit build and deploy pipelines to ensure integrity checks
  - Control plugin/extension usage and validate that they originate from trusted sources

A09:2025 – Logging & Alerting Failures

- Emphasises not just logging, but the alerting and monitoring side of security
- Failure implies difficulty to detect and respond to security incidents in a timely manner
- E.g. Lack of logging of critical events, inability to change log level in production, absence of alert on suspicious events, missing link between logs and incident response
- Checklist:
  - Ensure security events (authentication failures, privilege escalations, configuration changes) are logged reliably
  - Configure alerting rules (SIEM/IDS) to notify when suspicious patterns appear
  - Review retention and integrity of logs (immutable storage, audit trail)
  - Integrate logging monitoring into your DevSecOps pipeline (e.g., detect anomaly post deployment) rather than leaving it only to ops

A10:2025 – Mishandling of Exceptional Conditions

- A new category
- Focuses on how systems behave under abnormal conditions: exceptions, unexpected inputs, fail-open logic, error handling
  - Attackers increasingly exploit “edge cases” and “exception paths”
- E.g. Unhandled exceptions leading to stack traces disclosed, fallback logic that bypasses security controls, insecure default behavior on error, logic that grants access when an error occurs
- Checklist:
  - Audit exception handling flows
    - What happens when things go wrong? Are security controls bypassed inadvertently
  - Review fallback or default logic to avoid “secure by default” becoming “fail-to-open”
  - Include error and abnormal-state flows in threat modelling and test them (fault injection, chaos testing)
  - Ensure user-friendly errors don’t disclose sensitive internals (stack traces, system info)

Common Vulnerabilities and Exposures (CVE)

- CVE database: a glossary of all known vulnerabilities
  - Goal: Provide a standardized way of referencing to vulnerabilities
  - Maintained by MITRE Corporation (not-for-profit) in collaboration with international community
    - https://cve.mitre.org/cve/search_cve_list.html
  - Format "CVE-YYYY-NNNNN," where "YYYY" represents the year of assignment, and "NNNNN" is a sequential number

Common Weakness Enumeration (CWE)

- A list of software security weaknesses
  - Maintained by MITRE Corporation
  - Acts like a dictionary of vulnerabilities (types, not specific bugs)
- Describes what went wrong in code/design
  - Not an actual exploit → but a class of weakness
- Why is CWE Important?
  - Standard language for developers & security teams
  - Maps to real vulnerabilities like CVE

CWE ID	Name	What it Means	Example
CWE-79	XSS	Untrusted input rendered in browser	<script>alert(1)</script>
CWE-89	SQLi	Unsafe query construction	' OR 1=1 --
CWE-22	Directory Traversal	Accessing unintended files	../../etc/passwd
CWE-287	Auth Bypass	Weak/missing auth checks	Skipping login validation
CWE-200	Data Leak	Sensitive data disclosure	Stack trace with secrets

Aspect	CWE	CVE
Full Form	Common Weakness Enumeration	Common Vulnerabilities and Exposures
What it represents	Type of flaw	Specific bug in a system
Nature	Generic	Concrete
Example	CWE-79	CVE-2020-11022
Purpose	Identify root cause	Track real vulnerabilities
Used by	Developers, secure coding	Security teams, patching, alerts

Demo

Lets explore:

- https://cve.mitre.org/cve/search_cve_list.html
- https://www.cve.org/CVERecord?id=CVE-2021-44228 (highly exploited vulnerability)

https://www.ynytechnology.com.my/wp-content/uploads/2022/01/Picture2.png

https://media.licdn.com/dms/image/C5612AQHesRJDO6bG8A/article-cover_image-shrink_600_2000/0/1639594168677?e=2147483647&v=beta&t=RjopcdPQq2t9ohNz5VkPS4kj0-904DL6W-dH6k0MmoM

Common Vulnerability Scoring System (CVSS)

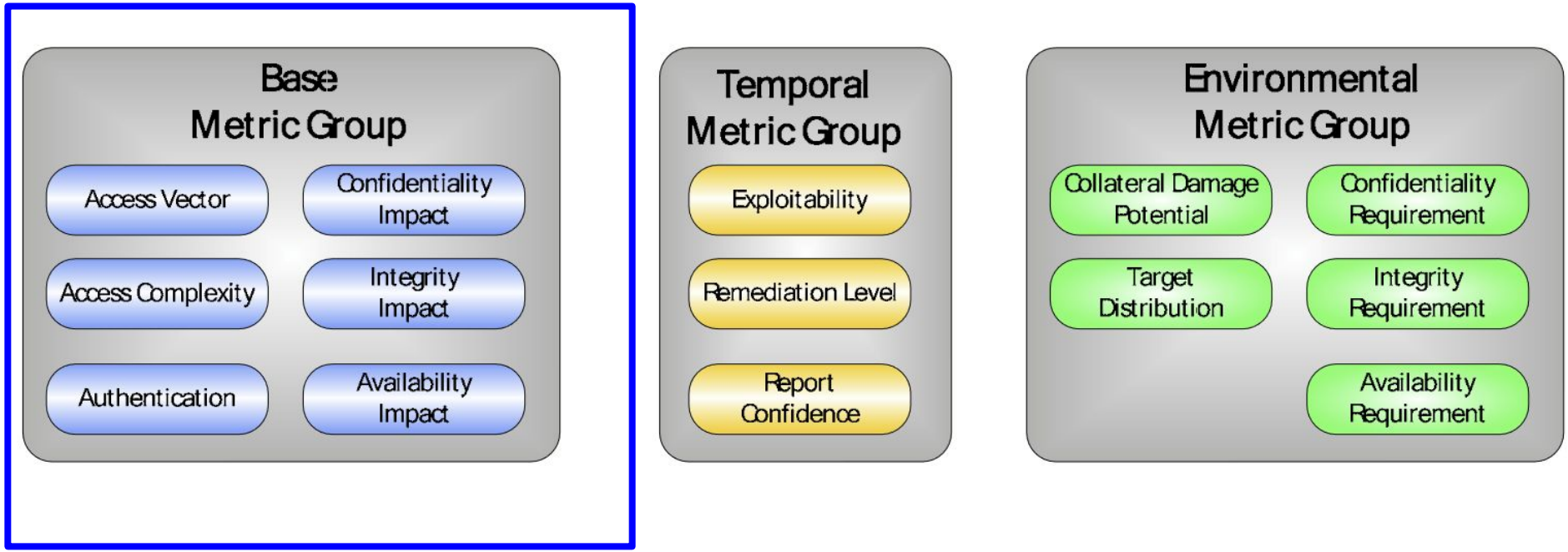
- In an organization, management needs to prioritize vulnerabilities
  - Remediate those that pose greatest risk
  - How to determine this?
- CVSS: standardized method for expressing characteristics and impact of vulnerabilities
  - How important a given vulnerability is in relation to other vulnerabilities?

- Common Vulnerability Scoring System-Special Interest Group (CVSS-SIG)
  - Forum of Incident Response and Security Teams (FIRST) sponsors and supports this group
  - Made of diverse group of security professionals that has a keen interest in this space!
  - Search URL: https://nvd.nist.gov/vuln/search
    - Enter CVE to get its score

All Tables and Figures hence forth taken from NIST publication: “Complete Guide to the Common Vulnerability Scoring System Version 2.0”

https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=51198

CVSS is composed of three metric groups:



- Base: represents intrinsic and fundamental characteristics of a vulnerability
  - Constant over time and user environments
  - How the vulnerability is accessed?
    - Dictated by Access Vector, Access Complexity, and Authentication metrics
  - How a vulnerability, if exploited, will directly affect an IT asset?
    - Measured by Impact metrics
    - Defined as the degree of loss of confidentiality, integrity, and availability
      - E.g. vulnerability could cause a partial loss of integrity and availability, but no loss of confidentiality

Access Vector

- Captures how the vulnerability is exploited?
- Local (L)
  - Requires local physical access to system or shell (e.g. Firewire/USB attacks)
- Adjacent Network (A)
  - Attacker have access to the broadcast domain of vulnerable software (e.g. local IP subnet, Ethernet segment or Bluetooth)
- Network (N)
  - Vulnerable software bound to network stak, remotely exploitable (e.g RPC buffer overflow)

Access Complexity

- High (H): Specialized access conditions exist
  - Elevated privileges, DNS hijacking capabilities, Victim must perform several atypical actions; Rare configuration of system; Narrow race condition window
- Medium (M): Somewhat specialized
  - Some info must be gathered before attack; affected configuration is non-default etc
- Low (L): Specialized conditions not needed
  - Default configuration, no skill or additional info; Winnable race condition

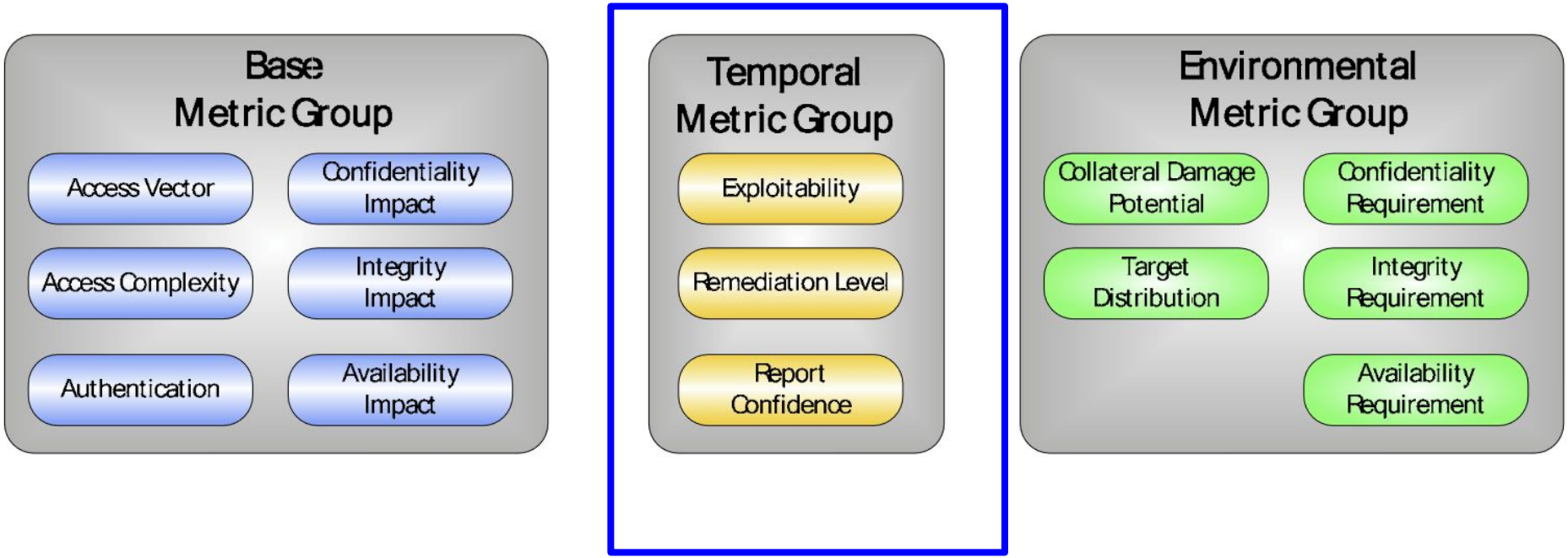
Authentication (Au)

- Number of times an attacker must authenticate to a target in order to exploit a vulnerability
  - Multiple (M)
    - Authenticate to OS, then application
  - Single (S)
  - None (N)

Confidentiality/Integrity/Availability Impact

- None (N)
- Partial (P): some information disclosed/modified or reduced performance (availability)
  - No full control over resource
    - e.g. some tables in database
  - Limited number of connections possible still!
- Complete (C): Total information disclosure/modification or total shutdown
  - Able to read/modify all system data (memory, files etc)
  - resource completely unavailable (availability)

CVSS is composed of three metric groups:



- Temporal: represents the characteristics of a vulnerability that change over time but not among user environments
  - This metric is optional

Exploitability

- Current state of exploit techniques or code availability

Metric Value	Description
Unproven (U)	No exploit code is available, or an exploit is entirely theoretical.
Proof-of-Concept (POC)	Proof-of-concept exploit code or an attack demonstration that is not practical for most systems is available. The code or technique is not functional in all situations and may require substantial modification by a skilled attacker.
Functional (F)	Functional exploit code is available. The code works in most situations where the vulnerability exists.
High (H)	Either the vulnerability is exploitable by functional mobile autonomous code, or no exploit is required (manual trigger) and details are widely available. The code works in every situation, or is actively being delivered via a mobile autonomous agent (such as a worm or virus).
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

Remediation Level

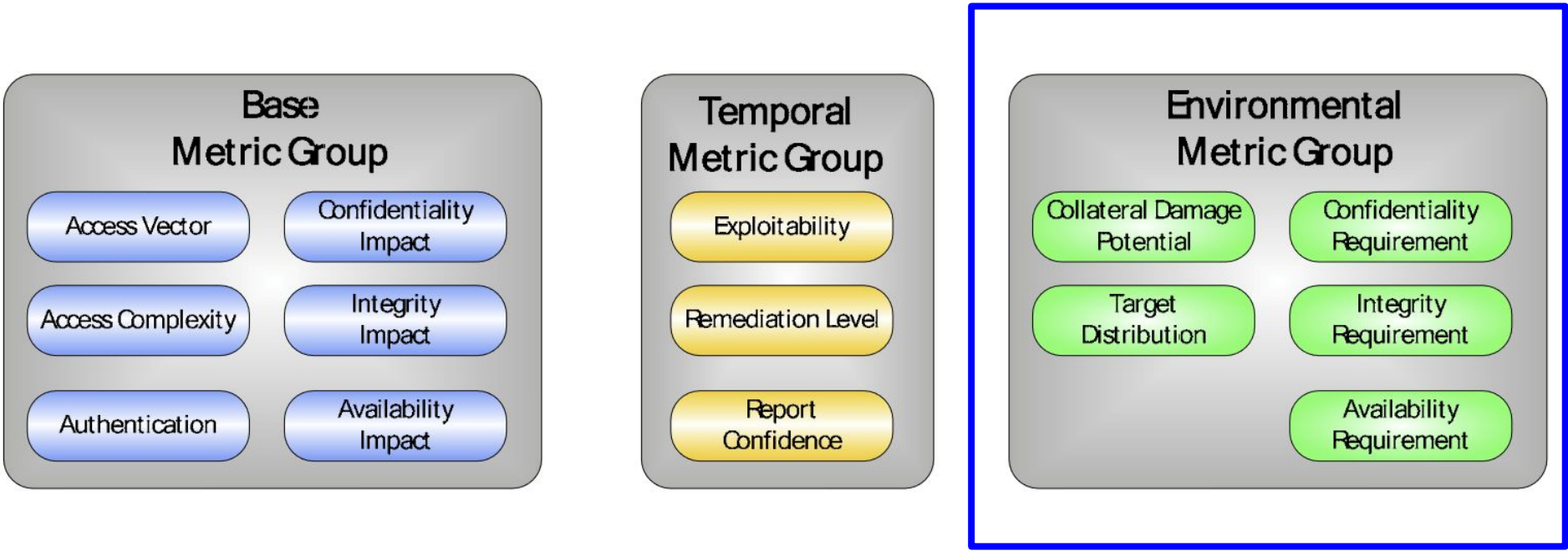
Metric Value	Description
Official Fix (OF)	A complete vendor solution is available. Either the vendor has issued an official patch, or an upgrade is available.
Temporary Fix (TF)	There is an official but temporary fix available. This includes instances where the vendor issues a temporary hotfix, tool, or workaround.
Workaround (W)	There is an unofficial, non-vendor solution available. In some cases, users of the affected technology will create a patch of their own or provide steps to work around or otherwise mitigate the vulnerability.
Unavailable (U)	There is either no solution available or it is impossible to apply.
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

Report Confidence

degree of confidence in the existence of the vulnerability and credibility of known technical details

Metric Value	Description
Unconfirmed (UC)	There is a single unconfirmed source or possibly multiple conflicting reports. There is little confidence in the validity of the reports. An example is a rumor that surfaces from the hacker underground.
Uncorroborated (UR)	There are multiple non-official sources, possibly including independent security companies or research organizations. At this point there may be conflicting technical details or some other lingering ambiguity.
Confirmed (C)	The vulnerability has been acknowledged by the vendor or author of the affected technology. The vulnerability may also be "Confirmed" when its existence is confirmed from an external event such as publication of functional or proof-of-concept exploit code or widespread exploitation.
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

CVSS is composed of three metric groups:



- Environmental: represents the characteristics of a vulnerability that are relevant and unique to a particular user’s environment
  - This is also optional!

Collateral Damage Potential (CDP)

Measure economic loss of productivity or revenue etc

Metric Value	Description
None (N)	There is no potential for loss of life, physical assets, productivity or revenue.
Low (L)	A successful exploit of this vulnerability may result in slight physical or property damage. Or, there may be a slight loss of revenue or productivity to the organization.
Low-Medium (LM)	A successful exploit of this vulnerability may result in moderate physical or property damage. Or, there may be a moderate loss of revenue or productivity to the organization.
Medium-High (MH)	A successful exploit of this vulnerability may result in significant physical or property damage or loss. Or, there may be a significant loss of revenue or productivity.
High (H)	A successful exploit of this vulnerability may result in catastrophic physical or property damage and loss. Or, there may be a catastrophic loss of revenue or productivity.
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

Target Distribution

Proportion of vulnerable systems

Metric Value	Description
None (N)	No target systems exist, or targets are so highly specialized that they only exist in a laboratory setting. Effectively 0% of the environment is at risk.
Low (L)	Targets exist inside the environment, but on a small scale. Between 1% - 25% of the total environment is at risk.
Medium (M)	Targets exist inside the environment, but on a medium scale. Between 26% - 75% of the total environment is at risk.
High (H)	Targets exist inside the environment on a considerable scale. Between 76% - 100% of the total environment is considered at risk.
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

Security Requirements (CR, IR, AR)

- Enables analyst to customize CVSS score depending on importance of affected IT asset to user's organization
  - Measured in terms of confidentiality, integrity, and availability
  - If availability is more important, can assign a greater value to availability, relative to confidentiality and integrity

Metric Value	Description
Low (L)	Loss of [confidentiality | integrity | availability] is likely to have only a limited adverse effect on the organization or individuals associated with the organization (e.g., employees, customers).
Medium (M)	Loss of [confidentiality | integrity | availability] is likely to have a serious adverse effect on the organization or individuals associated with the organization (e.g., employees, customers).
High (H)	Loss of [confidentiality | integrity | availability] is likely to have a catastrophic adverse effect on the organization or individuals associated with the organization (e.g., employees, customers).
Not Defined (ND)	Assigning this value to the metric will not influence the score. It is a signal to the equation to skip this metric.

Vectors

Metric Group	Vector
Base	AV:[L,A,N]/AC:[H,M,L]/Au:[M,S,N]/C:[N,P,C]/I:[N,P,C]/A:[N,P,C]
Temporal	E:[U,POC,F,H,ND]/RL:[OF,TF,W,U,ND]/RC:[UC,UR,C,ND]
Environmental	CDP:[N,L,LM,MH,H,ND]/TD:[N,L,M,H,ND]/CR:[L,M,H,ND]/ IR:[L,M,H,ND]/AR:[L,M,H,ND]

E.g. a vulnerability with

- Base metric values of “Access Vector: Low, Access Complexity: Medium, Authentication: None, Confidentiality Impact: None, Integrity Impact: Partial, Availability Impact: Complete” would have the following base vector: “AV:L/AC:M/Au:N/C:N/I:P/A:C.”

```
BaseScore[7] = round_to_1_decimal(((0.6*Impact)+(0.4*Exploitability)-1.5)*f(Impact))

Impact = 10.41*(1-(1-ConfImpact)*(1-IntegImpact)*(1-AvailImpact))

Exploitability = 20* AccessVector*AccessComplexity*Authentication

f(impact)= 0 if Impact=0, 1.176 otherwise

AccessVector     = case AccessVector of
                        requires local access: 0.395
                        adjacent network accessible: 0.646
                        network accessible: 1.0

AccessComplexity = case AccessComplexity of
                        high: 0.35
                        medium: 0.61
                        low: 0.71

Authentication   = case Authentication of
                        requires multiple instances of authentication: 0.45
                        requires single instance of authentication: 0.56
                        requires no authentication: 0.704
```

```
ConfImpact        = case ConfidentialityImpact of
                         none:              0.0
                         partial:           0.275
                         complete:          0.660

IntegImpact       = case IntegrityImpact of
                         none:              0.0
                         partial:           0.275
                         complete:          0.660

AvailImpact       = case AvailabilityImpact of
                         none:              0.0
                         partial:           0.275
                         complete:          0.660
```

```
TemporalScore = round_to_1_decimal(BaseScore*Exploitability
                *RemediationLevel*ReportConfidence)

Exploitability   = case Exploitability of
                       unproven:              0.85
                       proof-of-concept:      0.9
                       functional:            0.95
                       high:                  1.00
                       not defined:           1.00

RemediationLevel = case RemediationLevel of
                       official-fix:          0.87
                       temporary-fix:         0.90
                       workaround:            0.95
                       unavailable:           1.00
                       not defined:           1.00

ReportConfidence = case ReportConfidence of
                       unconfirmed:           0.90
                       uncorroborated:        0.95
                       confirmed:             1.00
                       not defined:           1.00
```

```
EnvironmentalScore = round_to_1_decimal((AdjustedTemporal+
(10-AdjustedTemporal)*CollateralDamagePotential)*TargetDistribution)

AdjustedTemporal = TemporalScore recomputed with the BaseScore's Impact sub-
equation replaced with the AdjustedImpact equation
```

```
AdjustedImpact = min(10,10.41*(1-(1-ConfImpact*ConfReq)*(1-IntegImpact*IntegReq)
                 *(1-AvailImpact*AvailReq)))

CollateralDamagePotential = case CollateralDamagePotential of
                                 none:            0
                                 low:             0.1
                                 low-medium:      0.3
                                 medium-high:     0.4
                                 high:            0.5
                                 not defined:     0

TargetDistribution        = case TargetDistribution of
                                 none:            0
                                 low:             0.25
                                 medium:          0.75
                                 high:            1.00
                                 not defined:     1.00

ConfReq          = case ConfReq of
                        low:             0.5
                        medium:          1.0
                        high:            1.51
                        not defined:     1.0

IntegReq         = case IntegReq of
                        low:             0.5
                        medium:          1.0
                        high:            1.51
                        not defined:     1.0

AvailReq         = case AvailReq of
                        low:             0.5
                        medium:          1.0
                        high:            1.51
                        not defined:     1.0
```

Example: CVE-2002-0392

```
-----------------------------------------------------
BASE METRIC                  EVALUATION         SCORE
-----------------------------------------------------
Access Vector                [Network]         (1.00)
Access Complexity            [Low]             (0.71)
Authentication               [None]            (0.704)
Confidentiality Impact       [None]            (0.00)
Integrity Impact             [None]            (0.00)
Availability Impact          [Complete]        (0.66)
-----------------------------------------------------
BASE FORMULA                                BASE SCORE
-----------------------------------------------------
Impact = 10.41*(1-(1)*(1)*(0.34)) == 6.9
Exploitability = 20*0.71*0.704*1 == 10.0
f(Impact) = 1.176
BaseScore = (0.6*6.9 + 0.4*10.0 - 1.5)*1.176
                                            == (7.8)
-----------------------------------------------------

-----------------------------------------------------
TEMPORAL METRIC              EVALUATION         SCORE
-----------------------------------------------------
Exploitability               [Functional]      (0.95)
Remediation Level            [Official-Fix]    (0.87)
Report Confidence            [Confirmed]       (1.00)
-----------------------------------------------------
TEMPORAL FORMULA                      TEMPORAL SCORE
-----------------------------------------------------
round(7.8 * 0.95 * 0.87 * 1.00)             == (6.4)
-----------------------------------------------------
```

```
ENVIRONMENTAL METRIC        EVALUATION         SCORE
-------------------------------------------------------
Collateral Damage Potential [None - High]  {0 - 0.5}
Target Distribution         [None - High]  {0 - 1.0}
Confidentiality Req.        [Medium]           (1.0)
Integrity Req.              [Medium]           (1.0)
Availability Req.           [High]             (1.51)
-------------------------------------------------------
ENVIRONMENTAL FORMULA            ENVIRONMENTAL SCORE
-------------------------------------------------------
AdjustedImpact = min(10,10.41*(1-(1-0*1)*(1-0*1)
         *(1-0.66*1.51))                    == (10.0)
AdjustedBase =((0.6*10)+(0.4*10.0)-1.5)*1.176
                                            == (10.0)
AdjustedTemporal == (10*0.95*0.87*1.0)       == (8.3)
EnvScore = round((8.3+(10-8.3)*{0-0.5})*{0-1})
                                       == (0.00 - 9.2)
-------------------------------------------------------
```

Overall CVSS Summary

- CVSS Base Score ranges from 0 to 10, with higher scores indicating higher severity
  - Base Score of 9.0 or above normally indicates critical vulnerability
  - Scores between 7.0 and 8.9 might be considered high severity
- Temporal and environmental metrics can further refine the score based on contextual factors
- Demo: https://nvd.nist.gov/vuln/search

Limitations

- Complexity and Overhead: Pretty complex calculations
  - Can be challenging for non-experts/small-organizations to understand and use effectively!
- Subjectivity in Scoring: different analysts or organizations may assign different scores to same vulnerability
  - Can lead to inconsistencies in risk assessments
- Each vulnerability is assessed independently, may give a false sense of security
- Not Designed for All Use Cases: Cannot help evaluate security of an entire system or network.

Summary

- Web development a popular profession
- Development without security consideration can lead to attacks
  - Many Server side and Client side attacks!
- OWASP Top 10 specifies rankings and remediation for top 10 attacks
- CVS is a database of all known vulnerabilities
- CVSS is an overall score (0-10) assigned to a given vulnerability.
  - Base Metric captures ease of attack and its impact on an IT asset
  - Temporal Metric captures impact over time
  - Environment Metric captures metrics relevant and unique to a particular user's environment

References

- OWASP TOP 10: https://owasp.org/www-project-top-ten/
- A Complete Guide to the Common Vulnerability Scoring System Version 2.0 : https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=51198